

Student Emotion Detection: Dual-Model (BiLSTM + BERT) Classroom Sentiment Analyzer

A Streamlit application comparing a custom BiLSTM classifier and a fine-tuned BERT model for real-time student emotion recognition

PyTorch Hugging Face Transformers Streamlit BiLSTM BERT scikit-learn

Project Description:

The Student Emotion Detection system is a Streamlit application that reads a short piece of free text from a student — a chat message, a forum post, or a quick check-in — and identifies the emotional state behind it, then responds with a short, supportive tip. The system runs two independent classifiers side by side on the same five emotion classes (**Bored**, **Confident**, **Confused**, **Curious**, **Frustrated**): a lightweight Bidirectional LSTM (BiLSTM) trained from scratch on a small custom classroom-emotion dataset, and a BERT-base model fine-tuned on the same labels.

For every prediction, the app displays both models' predicted emotion and confidence side by side, flags whether the two models agree, and falls back to the higher-confidence prediction as the "final emotion" whenever they disagree. The final emotion is paired with a short, encouraging suggestion (for example, "take a short break" for Bored, or "ask for a worked example" for Confused). Every prediction is logged to a session-history CSV, giving the app a simple built-in analytics dashboard and a searchable, filterable history log without needing a database.

Scenarios

- Scenario 1:** A student typing "I don't understand how this function works" during a coding lab submits the text through the Predict Emotion page. Both the BiLSTM and BERT models classify the input as **Confused** with high confidence, the app shows a green "models agree" banner, and offers the tip "Ask for a worked example or revisit the basics."
- Scenario 2:** A student writes "I have been trying for hours and nothing works" after a failed debugging session. Both models agree on **Frustrated**, and the guidance panel suggests relaxation techniques and taking the problem step by step.
- Scenario 3:** A student writes "This is the day for me because my friend insulted me in college." The BiLSTM model reads the sentence as **Bored** while BERT reads it as **Frustrated** — a genuine disagreement case captured live during testing (see Figure 4). The app surfaces a yellow "models disagree" warning showing both predictions and resolves the conflict by keeping the higher-confidence label as the final emotion.

Technical Architecture:



Figure 1: High-level architecture of the Student Emotion Detection application

Description: The Streamlit UI (app.py) is the single entry point for the user. When a student submits text, the UI sends the same raw string down two parallel pipelines: the BiLSTM pipeline (vocab_builder.py for tokenization, model.py for the network architecture, and the checkpoint saved by train.py) and the BERT pipeline (predict_bert.py, which loads the fine-tuned bert_model/ directory produced by train_bert.py). Both pipelines return an emotion label and a confidence score. A comparison layer in app.py checks whether the two labels agree, chooses a final emotion (the higher-confidence prediction when they disagree), and looks up a short guidance tip. The result is rendered to the user and appended as a new row in session_history.csv, which also powers the Analytics and Session History pages.

Note: This project does not use a relational database — state and history are persisted in flat files instead: the BiLSTM's vocabulary and weights are bundled into a single PyTorch checkpoint (bilstm_emotion_model.pt), the BERT model and tokenizer are saved as a Hugging Face model directory (bert_model/), and prediction history is appended to session_history.csv. If a future version introduces multi-user history behind a login, an ER diagram should be added here alongside this description.

Pre-requisites:

- **Python Programming Proficiency:** Python Documentation
- **PyTorch Fundamentals (tensors, nn.Module, training loops):** PyTorch Documentation
- **Hugging Face Transformers & Tokenizers:** Transformers Documentation
- **Streamlit Framework Knowledge:** Streamlit Documentation
- **scikit-learn (train/test split, label encoding):** scikit-learn Documentation
- **Pandas for tabular data handling:** Pandas Documentation
- **Version Control with Git:** Git Documentation

Project Workflow:

Milestone 1: Data Preparation and Vocabulary Building

- **Activity 1.1:** Collect and structure the labelled classroom-emotion dataset (dataset.csv), mapping text samples to five numeric emotion labels (0–4).
- **Activity 1.2:** Build a tokenizer and a word-to-index vocabulary from the dataset for the BiLSTM model.
- **Activity 1.3:** Prepare a BERT-compatible tokenizer configuration for the transformer pipeline.

Milestone 2: BiLSTM Model Development and Training

- **Activity 2.1:** Design the Bidirectional LSTM classifier architecture.
- **Activity 2.2:** Implement the training loop, loss function, and optimizer schedule.
- **Activity 2.3:** Save the trained weights, vocabulary, and model configuration together in a single checkpoint.

Milestone 3: BERT Model Fine-Tuning

- **Activity 3.1:** Load the dataset and encode emotion labels for sequence classification.
- **Activity 3.2:** Tokenize the text with BertTokenizer and build Hugging Face Dataset objects.
- **Activity 3.3:** Configure training arguments and fine-tune bert-base-uncased using the Trainer API.
- **Activity 3.4:** Save the fine-tuned model, tokenizer, and label classes for inference.

Milestone 4: Streamlit Application Development

- **Activity 4.1:** Load both models once at startup using st.cache_resource.
- **Activity 4.2:** Build the Predict Emotion page with side-by-side model cards, comparison logic, and guidance tips.
- **Activity 4.3:** Build the Analytics and Session History pages for reviewing past predictions.

Milestone 5: Integration, Testing and Deployment

- **Activity 5.1:** Wire the inference helpers (predict.py, predict_bert.py) into the Streamlit app.
- **Activity 5.2:** Diagnose and fix a label-ordering mismatch between the BERT label encoder and the emotion map.
- **Activity 5.3:** Run and manually test the app end-to-end, recording live predictions to session history.

Milestone 6: Conclusion

Milestone 1: Data Preparation and Vocabulary Building

This milestone focuses on turning raw labelled sentences into a form both models can consume: a simple integer vocabulary for the BiLSTM model, and a standard BERT tokenizer configuration for the transformer model.

Activity 1.1: Structure the Dataset

The dataset (dataset.csv) is a two-column CSV of text,label pairs, where label is an integer from 0 to 4 corresponding to Bored, Confident, Confused, Curious, and Frustrated respectively. Sample rows include entries such as "This chapter is very boring." (label 0) and "I am confident about tomorrow's coding test." (label 1).

Activity 1.2: Build the Tokenizer and Vocabulary

vocab_builder.py defines a lightweight tokenizer that lowercases text, preserves question marks, strips other punctuation, and splits on whitespace. It then walks the training CSV once to assign every unique token an integer index, reserving 0 for a padding token and 1 for unknown words:

```
def basic_tokenizer(text):
    text = text.lower().strip()
    text = re.sub(r'(\?)', r' \1 ', text)      # preserve question marks
    text = re.sub(r"[.,!_()];\"'\-]", "", text) # strip other punctuation
    return [token for token in text.split() if token]

def build_simple_vocab(csv_path):
    vocab = {"<pad>": 0, "<unk>": 1}
    next_index = 2
    # walk dataset.csv once, assigning new tokens the next free index
    ...
    return vocab
```

Activity 1.3: Prepare the BERT Tokenizer Configuration

The BERT pipeline reuses the standard bert-base-uncased WordPiece tokenizer (tokenizer_config.py, tokenizer.py) with a 512-token max length, lower-casing enabled, and the standard [CLS]/[SEP]/[PAD] special tokens, so no custom vocabulary needs to be built for the transformer model.

Milestone 2: BiLSTM Model Development and Training

Activity 2.1: Design the BiLSTM Architecture

model.py defines BiLSTMEmotionClassifier: an embedding layer feeding a 2-layer bidirectional LSTM, followed by dropout and a linear classification head over the five emotion classes. Variable-length sequences are packed with pack_padded_sequence, and the final forward and backward hidden states are concatenated before the classifier:

```
self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
self.lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers=2,
                    bidirectional=True, batch_first=True, dropout=dropout_prob)
self.fc = nn.Linear(hidden_dim * 2, output_dim)

def forward(self, text, text_lengths):
    embedded = self.dropout(self.embedding(text))
    packed = nn.utils.rnn.pack_padded_sequence(
        embedded, text_lengths, batch_first=True, enforce_sorted=False)
    _, (hidden, _) = self.lstm(packed)
    hidden = torch.cat((hidden[-2], hidden[-1]), dim=1)  # fwd + bwd
    return self.fc(self.dropout(hidden))
```

Activity 2.2: Implement the Training Loop

train.py trains the model for 35 epochs with the Adam optimizer (lr = 0.003, weight decay 1e-4), a step-based learning-rate scheduler that halves the rate every 15 epochs, gradient clipping at norm 1.0, and cross-entropy loss over single-example batches:

```
EMBEDDING_DIM = 64
HIDDEN_DIM = 64
OUTPUT_DIM = 5
EPOCHS = 35
LEARNING_RATE = 0.003

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE, weight_decay=1e-4)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=15, gamma=0.5)

for epoch in range(EPOCHS):
    for sentence, label in zip(sentences, labels):
        outputs = model(sentence.unsqueeze(0), lengths)
        loss = criterion(outputs, targets)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
```

Activity 2.3: Save the Model, Vocabulary, and Config Together

To avoid any mismatch between a saved checkpoint and the vocabulary or architecture it was trained with, all three are written into a single file, bilstm_emotion_model.pt:

```
torch.save({
    "model_state_dict": model.state_dict(),
    "vocab": vocab,
    "config": {
        "vocab_size": len(vocab),
        "embedding_dim": EMBEDDING_DIM,
        "hidden_dim": HIDDEN_DIM,
        "output_dim": OUTPUT_DIM,
        "dropout_prob": DROPOUT
    }
}, "bilstm_emotion_model.pt")
```

Milestone 3: BERT Model Fine-Tuning

Activity 3.1: Load the Dataset and Encode Labels

`train_bert.py` reads the same `dataset.csv`, maps the numeric labels to their emotion names through an `EMOTION_MAP`, and re-encodes those names with scikit-learn's `LabelEncoder` so the class order used by BERT is captured and saved alongside the model:

```
EMOTION_MAP = {0: "Bored", 1: "Confident", 2: "Confused", 3: "Curious", 4: "Frustrated"}

df["label"] = df["label"].astype(int).map(EMOTION_MAP)
label_encoder = LabelEncoder()
df["label"] = label_encoder.fit_transform(df["label"])
np.save("label_classes.npy", label_encoder.classes_)
```

Activity 3.2: Tokenize and Build Hugging Face Datasets

Text is split 80/20 into train and test sets (stratified by label), tokenized with `BertTokenizer` at a max length of 128 with fixed padding, and wrapped in Hugging Face Dataset objects formatted as PyTorch tensors.

Activity 3.3: Configure Training and Fine-Tune

The model is fine-tuned for 10 epochs with a linear learning-rate schedule and a 10% warmup ratio, evaluating and checkpointing after every epoch and keeping the best checkpoint by accuracy:

```
training_args = TrainingArguments(
    output_dir="bert_output",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    num_train_epochs=10,
    warmup_ratio=0.1,
    lr_scheduler_type="linear",
    load_best_model_at_end=True,
    metric_for_best_model="accuracy"
)
```

Activity 3.4: Save the Model, Tokenizer, and Label Classes

```
trainer.save_model("bert_model")
tokenizer.save_pretrained("bert_model")
```

Milestone 4: Streamlit Application Development

Activity 4.1: Load Both Models Once at Startup

app.py wraps both model-loading functions in @st.cache_resource so the BiLSTM checkpoint and the BERT model/tokenizer are only loaded into memory once per server session, not on every user interaction:

```
@st.cache_resource
def load_bilstm():
    checkpoint = torch.load("bilstm_emotion_model.pt", map_location=DEVICE)
    model = BiLSTMEmotionClassifier(**checkpoint["config"])
    model.load_state_dict(checkpoint["model_state_dict"])
    return model, checkpoint["vocab"]

@st.cache_resource
def load_bert():
    return load_bert_components()
```

Activity 4.2: Build the Predict Emotion Page

The main page takes a free-text input, runs it through both predict_student_emotion (BiLSTM) and predict_student_emotion_bert (BERT), and renders each result in its own colour-coded card. It then compares the two labels, shows an agree/disagree banner, picks the higher-confidence prediction as the final emotion, looks up a guidance tip, and logs the full result to session_history.csv:

```
final_emotion = bert_emotion if bert_confidence > confidence else emotion

for tip in GUIDANCE_TEXT.get(final_emotion, ["Keep learning.']):
    st.write("\u2705", tip)

log_prediction(
    input_text=user_input,
    bilstm_emotion=emotion, bilstm_confidence=confidence,
    bert_emotion=bert_emotion, bert_confidence=bert_confidence,
    final_emotion=final_emotion
)
```

Activity 4.3: Build the Analytics and Session History Pages

The Analytics page reads session_history.csv and shows the latest prediction as a set of metrics plus three side-by-side bar charts (BiLSTM, BERT, and final emotion). The Session History page presents the full log as a filterable, searchable table (by final emotion and by free text), with buttons to download the filtered history as CSV or clear it entirely.

Milestone 5: Integration, Testing and Deployment

Activity 5.1: Wire Up the Inference Helpers

`predict.py` handles BiLSTM inference: it tokenizes the input, maps unknown words to the <unk> index, runs the model, and returns the predicted emotion, confidence, and a full probability breakdown for all five classes. `predict_bert.py` mirrors this for the transformer model, tokenizing with the saved BERT tokenizer and applying softmax to the model's logits. Both functions defensively return an "Error" result on exception rather than letting the Streamlit page crash.

Activity 5.2: Fix a Label-Ordering Mismatch

During testing, BERT's predicted emotion labels did not line up with the BiLSTM's labels for the same input, even when the underlying class index was correct. The root cause was that `label_classes.npy` stored scikit-learn's **alphabetically sorted** class order rather than guaranteeing the exact order used elsewhere in the app. `fix_label_classes.py` was written to explicitly rebuild `label_classes.npy` from the same `EMOTION_MAP` used everywhere else, backing up the old file first:

```
EMOTION_MAP = {0: "Bored", 1: "Confident", 2: "Confused", 3: "Curious", 4: "Frustrated"}

old_classes = np.load(LABEL_FILE, allow_pickle=True)
np.save(LABEL_FILE + ".backup", old_classes)  # back up before overwriting

correct_classes = np.array([EMOTION_MAP[i] for i in range(len(EMOTION_MAP))], dtype=object)
np.save(LABEL_FILE, correct_classes)
```

After running this fix and restarting the app, BERT's label indices and the BiLSTM's labels refer to the same emotion names in the same order.

Activity 5.3: Run and Test the Application

- Install dependencies: `streamlit`, `torch`, `transformers`, `pandas`, `numpy`, `scikit-learn`.
- Ensure `bilstm_emotion_model.pt`, the `bert_model/` directory, and `label_classes.npy` are present in the project root.
- Launch the app locally: `streamlit run app.py`, then open `http://localhost:8501`.
- Manually test a range of inputs across all five emotions, including ambiguous sentences, and confirm both models respond, agree/disagree correctly, and the session history logs every entry (see live results below).

Results & Observations

The table below is computed directly from a live testing session logged to `session_history.csv` while manually exercising the app (13 predictions):

Metric	Value
Total predictions logged	13
Models agreed	5 / 13 (38.5%)
Average BiLSTM confidence	99.67%
Average BERT confidence	92.44%

Observation: Across this test session the BiLSTM model was almost always near 100% confident in its prediction — even on the inputs where it disagreed with BERT and, by inspection, appeared to be wrong (for example, reading "I am sad because my friend not answer my phone" as *Frustrated* at 99.7% confidence, or a revenge-themed sentence as *Curious* at 99.9% confidence). This is a classic symptom of a small BiLSTM trained from scratch on a very small dataset: the softmax output saturates and the confidence score stops being a reliable indicator of correctness. BERT's confidence, averaging 92.4% and varying more per example, tracked correctness noticeably better in this sample. This suggests two follow-ups for a future iteration: (1) grow the training dataset well beyond its current size, and (2) treat BiLSTM confidence with caution, or calibrate it (e.g. temperature scaling), rather than using it at face value when the two models disagree.

Predict Emotion Page — Input

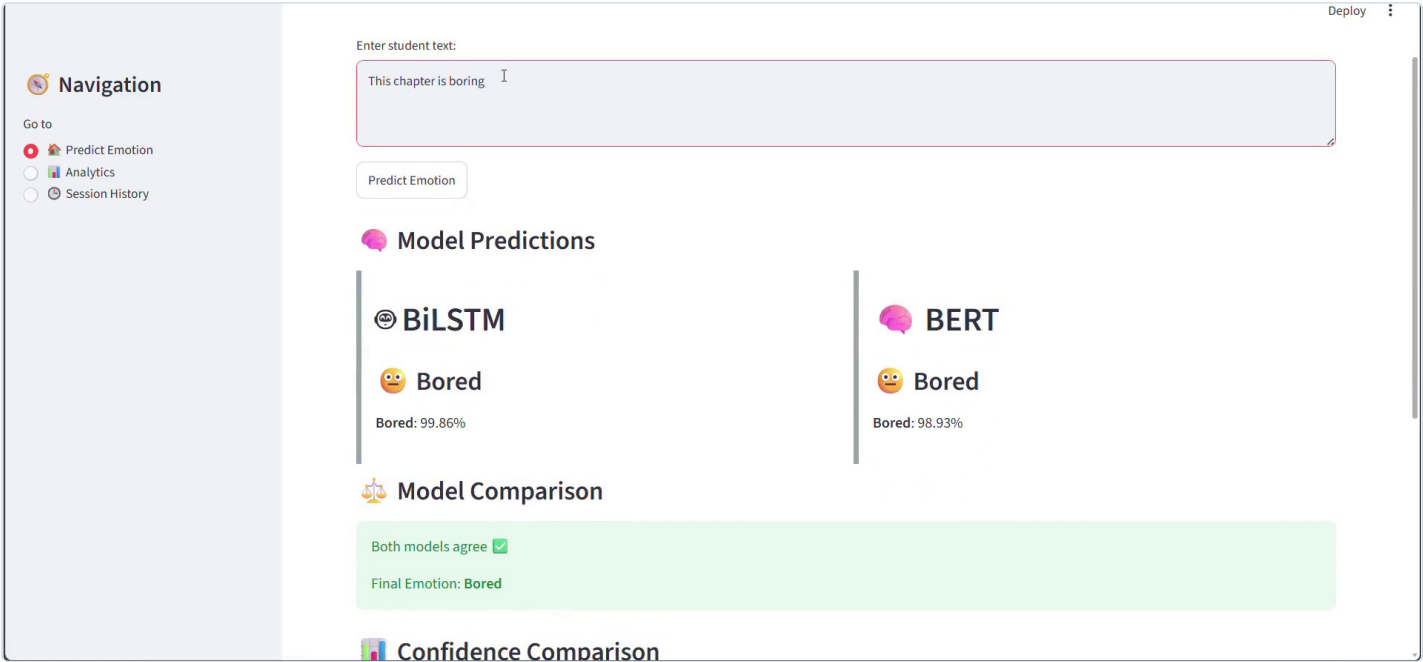


Figure 2: A student's text being entered on the Predict Emotion page during live testing.

Description: The default landing page. A student types free text into a text area and clicks "Predict Emotion." The sidebar (left) provides navigation between the Predict Emotion, Analytics, and Session History pages.

Predict Emotion Page — Model Predictions

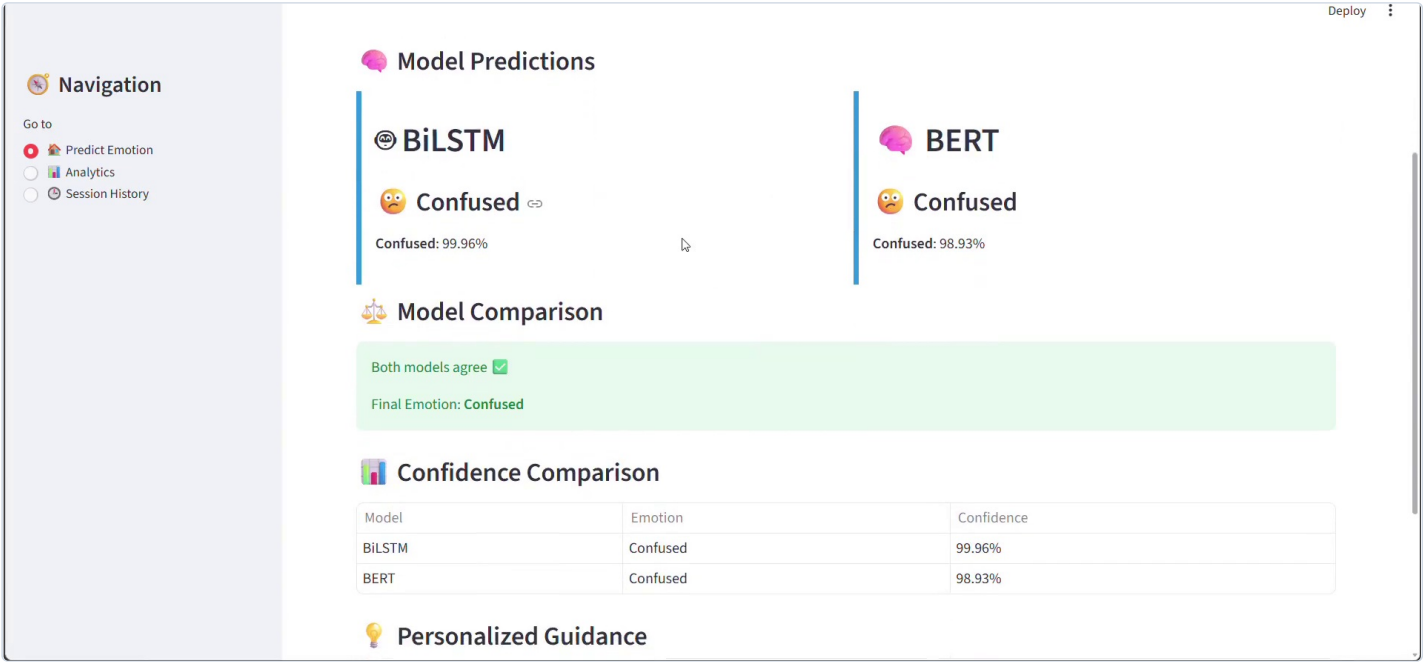


Figure 3: Both models' predictions rendered as side-by-side cards after clicking "Predict Emotion."

Description: The results section shows two colour-coded cards side by side — one for the BiLSTM prediction, one for BERT — each displaying the predicted emotion and a confidence percentage.

Predict Emotion Page — Disagreement Case

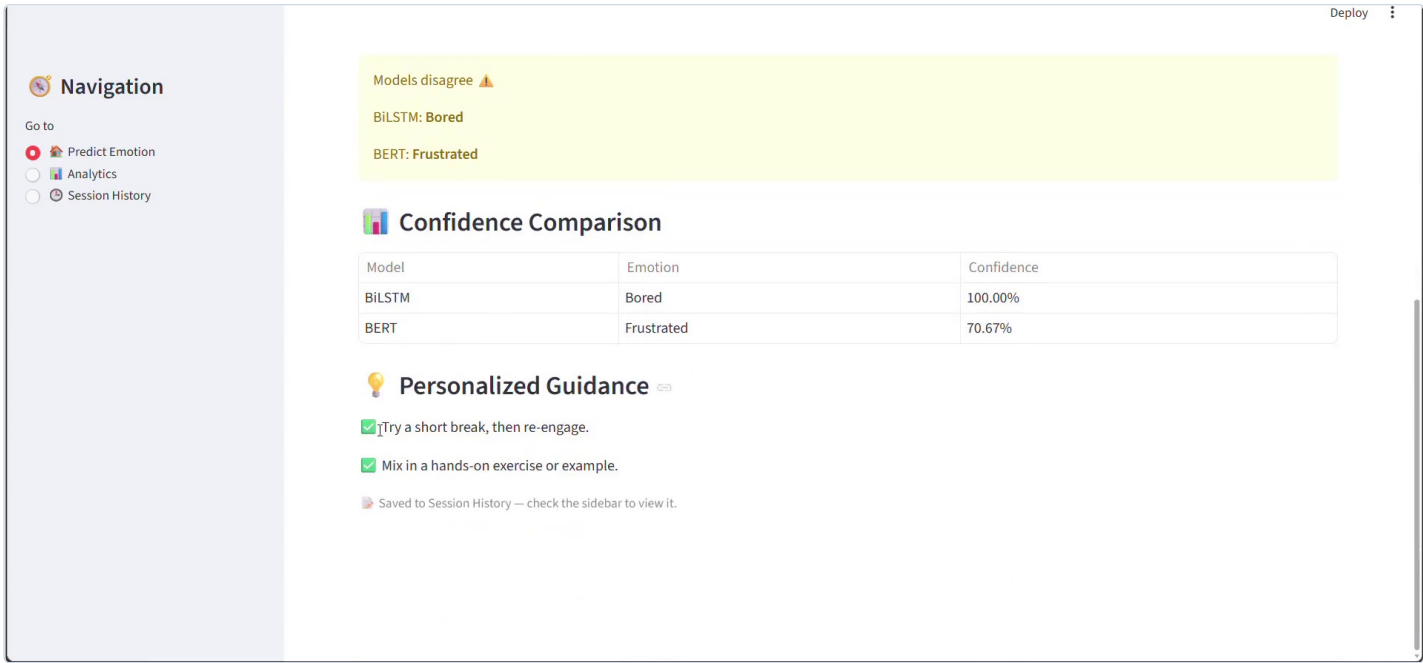


Figure 4: A real disagreement captured live — the two models predict different emotions for the same sentence.

Description: Below the prediction cards, a comparison banner confirms agreement (green) or, as shown here, flags a disagreement (yellow/red) with both labels displayed side by side. The app still resolves a final emotion using the higher-confidence prediction and logs the disagreement to session history for later review.

Analytics Page

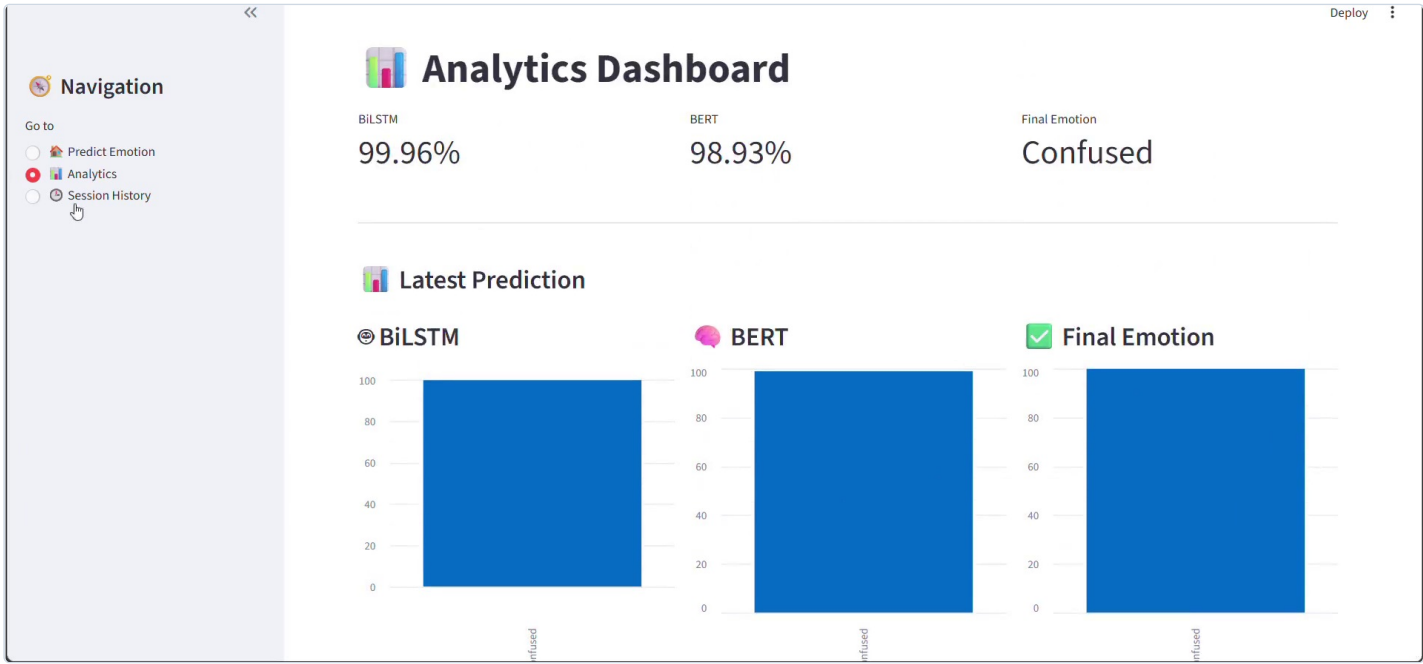


Figure 5: The Analytics dashboard, showing the latest prediction's confidence for BiLSTM and BERT.

Description: Reads the session history CSV and summarizes the most recent prediction as metric tiles and bar charts comparing BiLSTM, BERT, and final-emotion confidence. If no predictions have been made yet, the page shows a simple "No predictions yet" message instead.

Session History Page

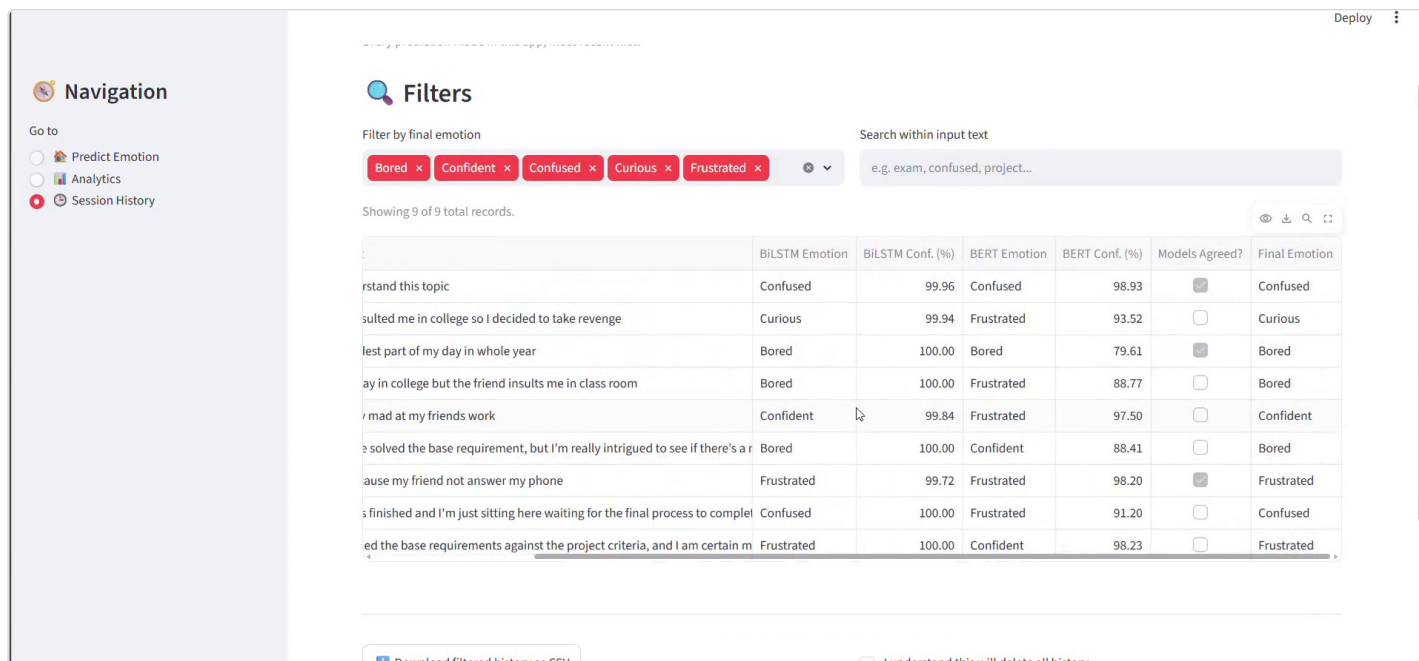


Figure 6: The filterable Session History table, listing every prediction made during testing.

Description: Displays every logged prediction, most recent first, in a sortable table with columns for timestamp, input text, both models' emotions and confidences, an agreement flag, and the final emotion. Filter controls let the user narrow the table by final emotion or search within the input text, and action buttons allow downloading the filtered rows as CSV or clearing the history.

Conclusion:

The Student Emotion Detection application demonstrates how a lightweight, from-scratch BiLSTM classifier and a fine-tuned BERT model can be run side by side to cross-check predictions on the same emotion-classification task, rather than relying on a single model's output. Built with Streamlit, the app turns raw student text into an emotion label, a confidence score, and a supportive next step in seconds, while its CSV-based session history doubles as a lightweight analytics tool for reviewing trends and auditing disagreements between the two models. Live testing surfaced two real findings worth carrying into the next iteration: a label-ordering bug between the two pipelines (since fixed), and a tendency for the small BiLSTM model to be overconfident relative to BERT, with the two models agreeing on only 38.5% of tested inputs. Looking ahead, the system would benefit most from a larger and more diverse training dataset, confidence calibration for the BiLSTM model, multi-language support, and a proper database backend for multi-user history and authentication — evolving the project from a working prototype into a classroom-ready engagement tool.